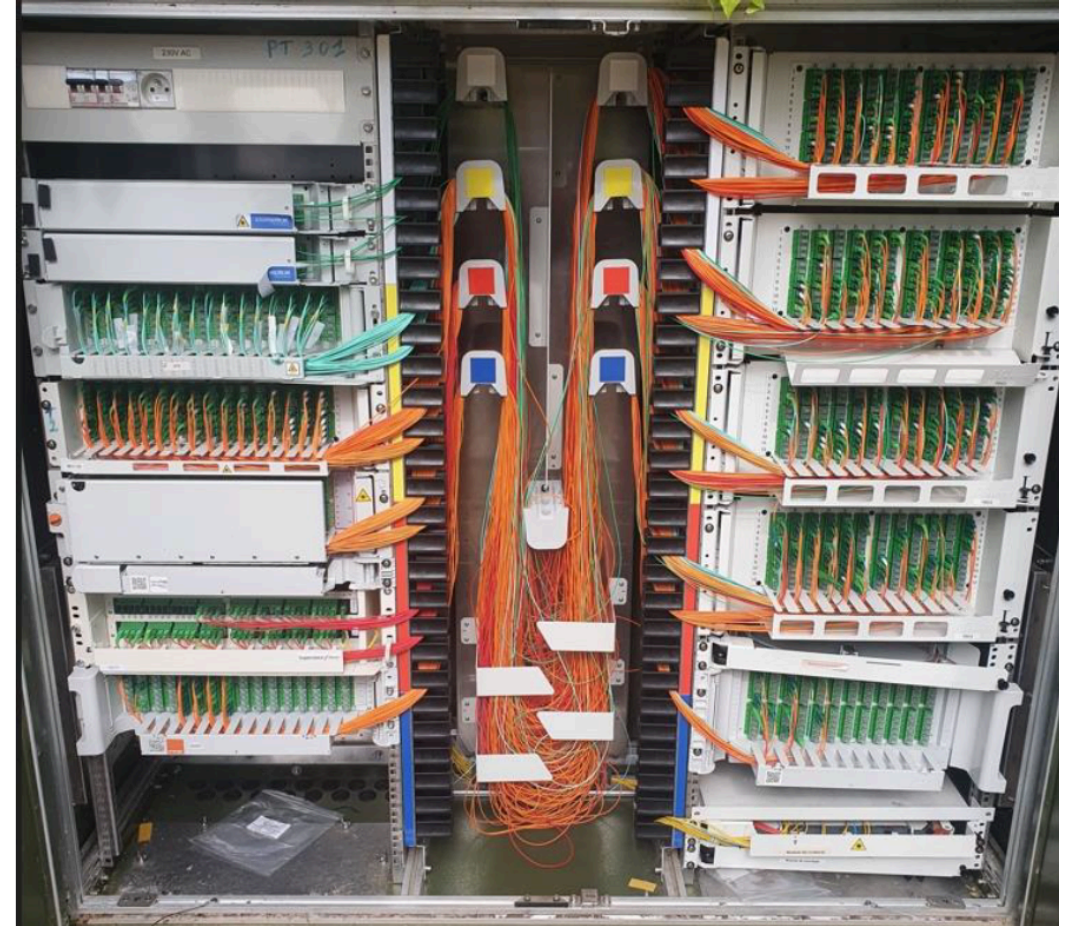
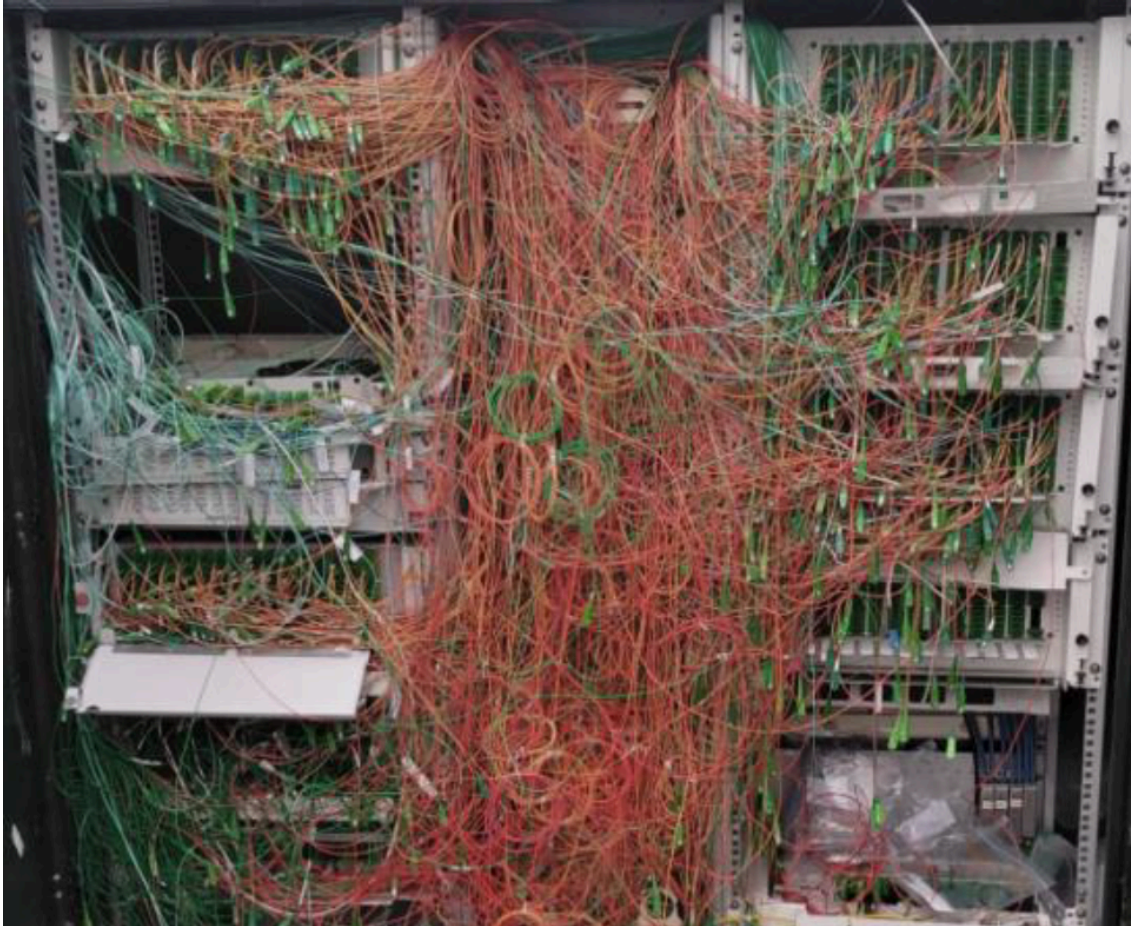


Développement d'applications & TMA

Animé par Sylvain Labasse

INTRODUCTION



AU TERME DU MODULE, VOUS SAUREZ...

Organiser une équipe de dév. pour la maintenance

Utiliser des outils adaptés pour assurer la qualité

Prévenir les régressions via les tests fonctionnels

Sécuriser la supply chain par une approche DevSecOps

Consolider progressivement l'application

360 LEARNING

Supports

Copie des slides + Notes

Ateliers

Evaluation

4 ateliers sur 4

0	1	2	3	4
Non remis	Hors sujet	<	=	>
Attentes				

DÉVELOPPEMENT D'APPLICATIONS ET TMA

Projets de maintenance

- Contractualisation

- Acteurs

- Approches

Outillage

- Ticketing

- Livraison

- Métriques

DÉVELOPPEMENT D'APPLICATIONS ET TMA

Tests

- Plan de test

- Fonctionnels

- Playwright

DevSecOps

- Modélisation des menaces

- Analyse statique

- Supply Chain

PROJETS DE MAINTENANCE

OBJECTIFS

Spécificités des projets de maintenance informatique

Aspect juridique et clauses fréquentes

Organisation et fonctionnement des équipes

Différences agiles / conventionnel

PROJETS DE MAINTENANCE

→ Contractualisation

Acteurs

Approches

Synthèse

CONTRAT DE MAINTENANCE

Type

Partie : Développeur original du projet / Tiers : TMA

Nature : Forfait, délégation, mixte

Périmètre : Corrective, Evolutive, Préventive

Termes

Niveau de service : SLA, SLO, pénalités

Autres : Durée (annuel), NDA, paiement, ...

JURIDIQUE

Contrat informatique

Obligation de moyen : « règles de l'art »

Formalisation de méthode de travail

Précision des termes - Glossaire

Propriété des codes et documentations / Réversibilité

RGPD

Données personnelles maintenues/manipulées

Clauses spécifiques

BUDGET

Estimation = Temps mensuel / Compétences

Maintenance = 10%/an du montant du projet de dév.

≠ pour TMA : Estimation du temps / compétences requises

$\Sigma \text{TJM}_{\text{compétence}} \times \text{Nombre de jours estimés}$

Suivi

Niveau de support : 1 (base), 2 (avancé), 3 (expert), 4 (externe)

Tickets : Temps, Nombre

Tableau de suivi financier => rentabilité

PROJETS DE MAINTENANCE

✓ Contractualisation

→ Acteurs

Approches

Synthèse

ACTEURS

MOA

Propriétaire de l'application à maintenir

Remonte les anomalies, demandes d'amélioration, priorités, ...

Eventuellement assisté : AMOA

MOE

Effectue les corrections et modifications

Responsable qui dispatch et valide les corrections + équipe

Niveau de sous-traitance à préciser

PROJETS DE MAINTENANCE

✓ Contractualisation

✓ Acteurs

→ Approches

Synthèse

INVARIANTS

Outils

DevOps : Dépôt, Test, CI/CD, Documentation

Ticketing pour la remontée et le suivi des anomalies

Communication

Feedback permanent – Correction, (in)efficacité

Information sur les versions

Conflits – Prévention, gestion

CASCADE

Prévention, évolutions

Rédaction de spéc. fonctionnelles

Implémentation après validation

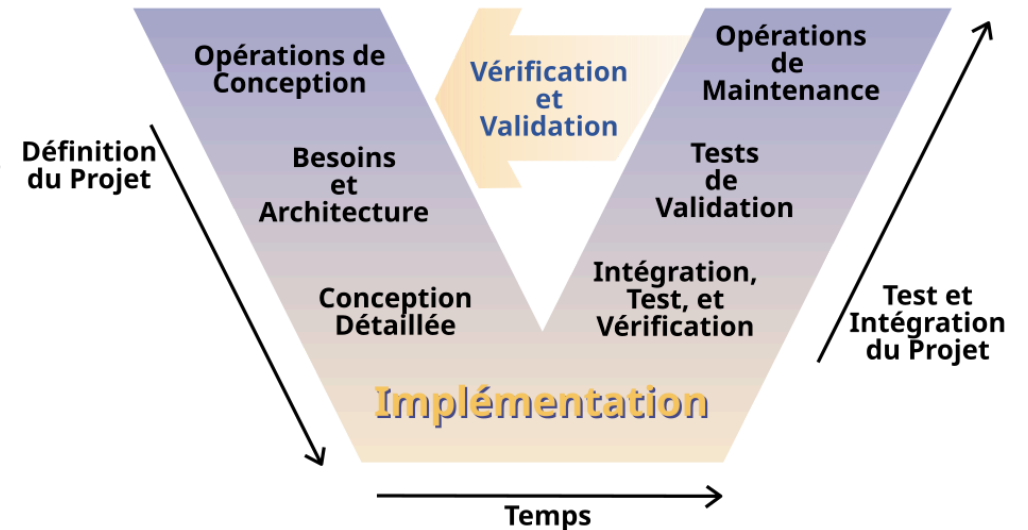
Tests unitaires/fonctionnels

Corrections

Absent des spécifications => Pas à corriger

Anomalie non-bloquante : Accès détourné à la fonctionnalité

Anomalie bloquante : Aucun accès à la fonctionnalité



AGILE

Backlog

Récits pour prévention, évolution

Orientation produit ou Livraison à chaque sprint

Consolidation

Daily-scrum favorise le partage d'expérience

Intégration continue nécessaire

XP : Pair programming, refactoring, tests

PROJETS DE MAINTENANCE

- ✓ Contractualisation
- ✓ Acteurs
- ✓ Approches
- Synthèse

RÉSUMÉ

Spécificités des projets de maintenance informatique

Aspect juridique et clauses fréquentes

Organisation et fonctionnement des équipes

Différences agiles / conventionnel

OUTILLAGE

OBJECTIFS

Liste de solutions utilisables pour la maintenance

Configurations adaptées à la maintenance application

Importance de la traçabilité pour l'optimisation

Mesures et indicateurs de qualité

OUTILLAGE

→ Ticketing
Livraison
Métriques
Synthèse

TICKETING

Fonctions

Portail de saisie des anomalies

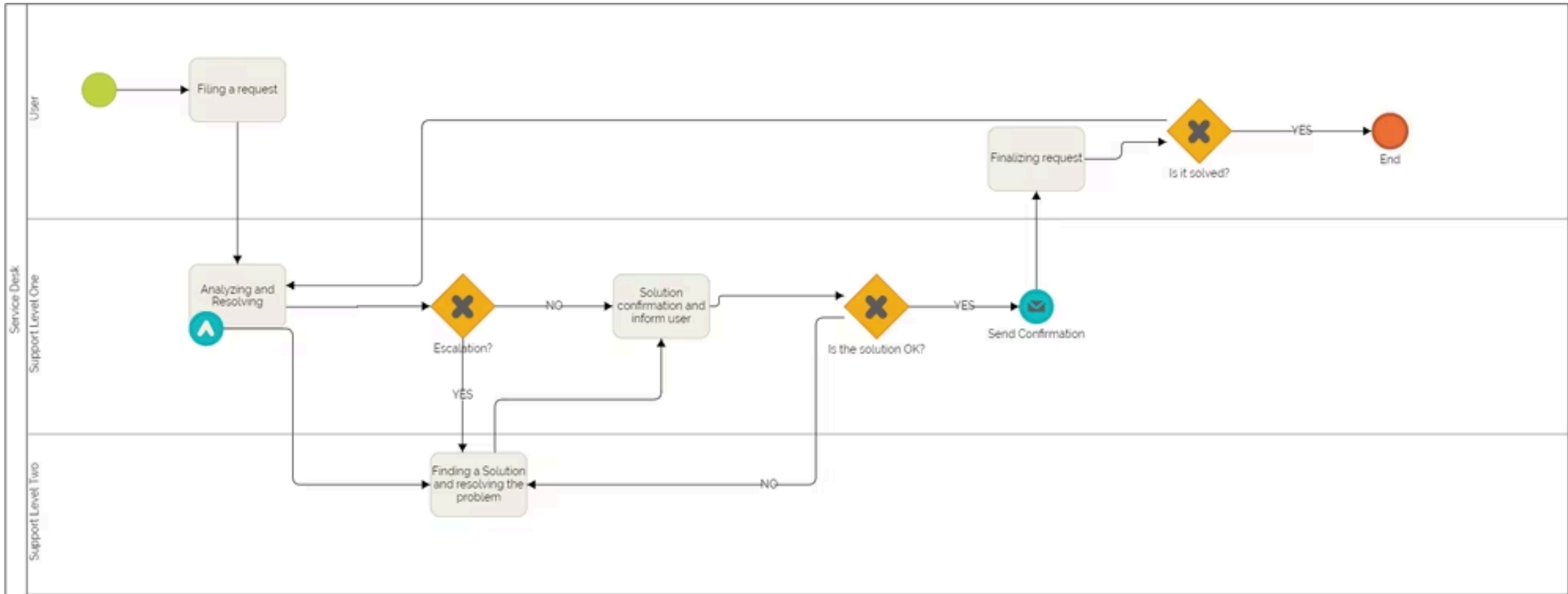
Workflow : Affectation, suivi, alertes, ...

Help Desk

Projets/Crm : Jira, Zoho, HubSpot, Glpi, ...

Pure player : Zendesk, Freshdesk (AI), ...

WORKFLOW



OUTILLAGE

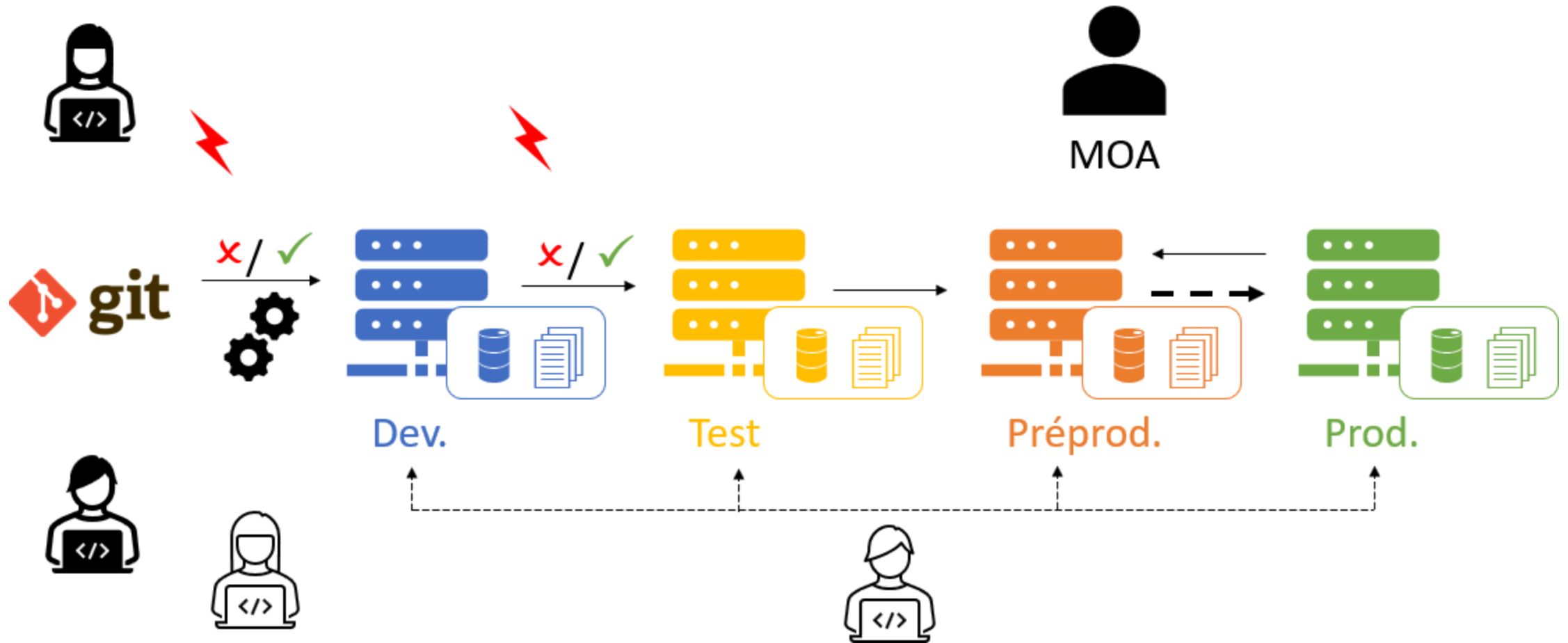
✓ Ticketing

→ Livraison

Métriques

Synthèse

LIVRAISON



CI/CD SUR SITE

Jenkins

Polyvalent : Open source / Serveur
Extensible
Connaissances système

Autres

TeamCity (JetBrains)

Bamboo (Atlassian)

Pure player : CircleCI, GitLab

	 Jenkins	 TeamCity	 circleci	 Bamboo	 GitLab
Open source	Yes	No	No	No	No
Ease of use & setup	Medium	Medium	Medium	Medium	Medium
Built-in features	3/5	4/5	4/5	4/5	4/5
Integration	★★★★★	★★★	★★★★	★★★	★★★★
Hosting	On premise & Cloud	On premise & Cloud	On premise	On premise & Bitbucket as Cloud	On premise & Cloud
Free version	Yes	Yes	Yes	Yes	Yes
Build agent license pricing	Free	From \$59 per month	From \$15 per month	From \$10 one-off payment	From \$19 per month per user
Supported OSs	Windows, Linux, macOS, Unix-like OS	Linux or MacOS	Windows, Linux, macOS, Solaris, FreeBSD and more	Windows, Linux, macOS, Solaris	Linux distributions: Ubuntu, Debian, CentOS, Oracle Linux

CI/CD INTÉGRÉ AU DÉPÔT

Offre

- Liaison simple aux versions
- Fichiers de configuration
- Tarification au temps CPU

Solutions

- GitHub : Microsoft
- BitBucket : Atlassian
- GitLab : GitLab Inc

OFFRES CI/CD

Offres Cloud

Déploiement facilité : Sources et infra au même endroit

Scalabilité et intégré au budget cloud

Azure DevOps, Amazon CodePipeline, Google Cloud Build

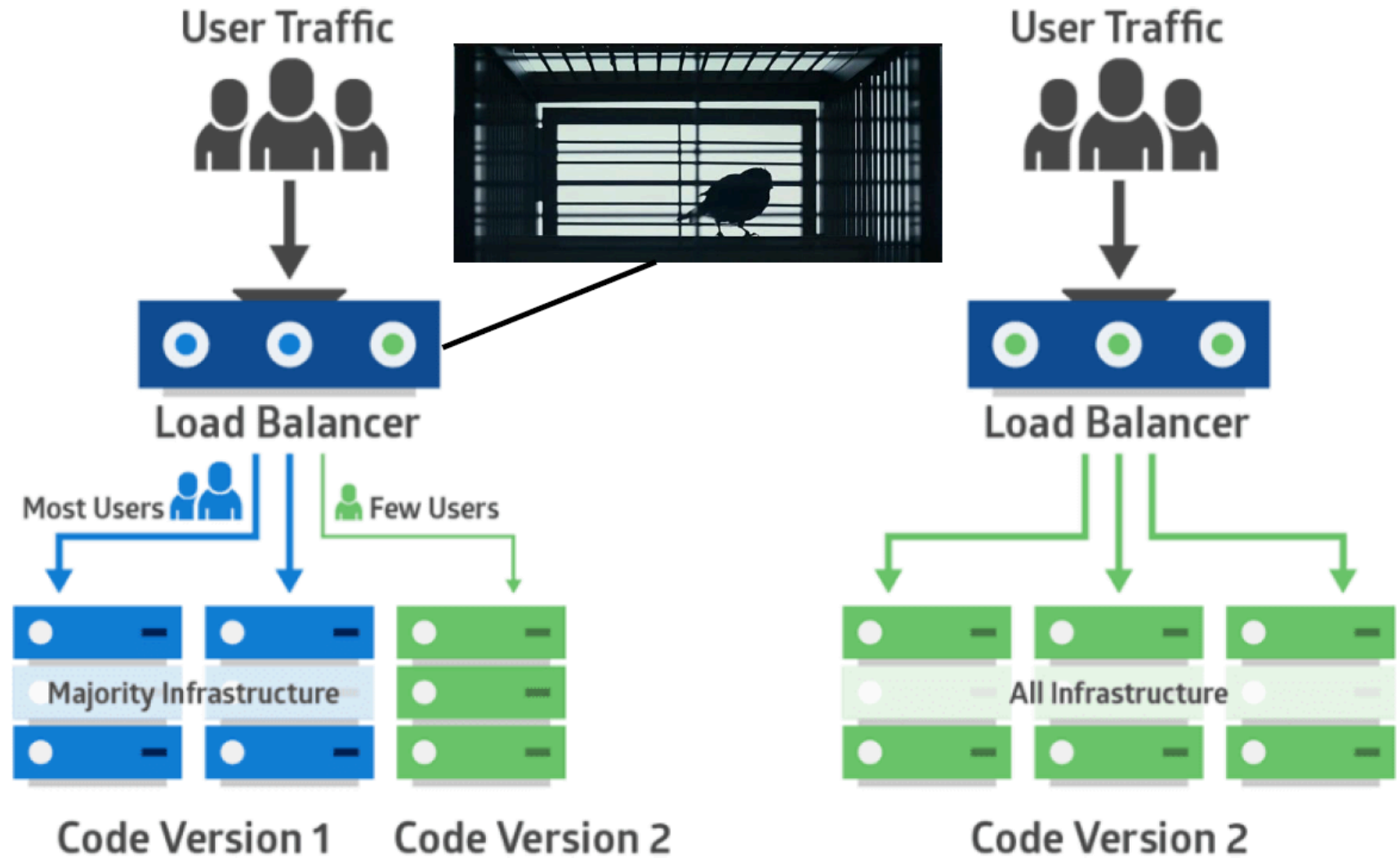
Saas – PurePlayers

Circle CI

Travis CI

Jetbrains Team City

LIVRAISON CANARI



OUTILLAGE

✓ Ticketing

✓ Livraison

→ Métriques

Synthèse

« Quand une mesure devient un objectif,
elle cesse d'être une bonne mesure »

Loi de Goodhart

INDICATEURS

Basé sur le code/tests

- Taille / Complexité

- Redondance : Lignes, blocs (séquence n instructions)

- Couverture

Basés sur les tickets : Nouveau, temps passé, ...

- Maintenabilité – Sur nouvelle fonctionnalité / modification

- Fiabilité – Sur une fonctionnalité existante

- Sécurité – Sur vulnérabilité

OUTILS

SonarQube - Référence

Version communautaire ou hébergée

Trentaine de langages

Analyse statique, hautement paramétrable

Autres

Codacy : Sans CI, sécurité, ...

DeepSource : Sans CI, fix par AI, ...

Github Super Linter – Workflow Github de validation du code

Checkmarx : Sécurité

OUTILLAGE

- ✓ Ticketing
- ✓ Livraison
- ✓ Métriques
- Synthèse

RÉSUMÉ

Liste de solutions utilisables pour la maintenance

Configurations adaptées à la maintenance application

Importance de la traçabilité pour l'optimisation

Mesures et indicateurs de qualité

TESTS

OBJECTIFS

Utilité des tests pour la maintenabilité

Construction et contenu d'un plan de test

Outils de tests fonctionnels selon le type d'appli.

Prise en main de Playwright pour le web et les APIs

TESTS

→ Plan de test
Fonctionnels
Playwright
Synthèse

PLAN DE TEST

Fonctionnalité	Cas	Séquence
Recherche	Mot-Clé – 5 résultats (usuel) Plusieurs mots-clés – 2 résultats (usuel) Mot-Clé – Pas de résultat (extrême) (erreur)	Arrange, Act, Assert
Inscription	S.Labasse / labasse+nouv@gmail.com non inscrit (usuel) S.Labasse / labasse@gmail.com déjà inscrit (extrême) S. / labasse+nouv@gmail.com (erreur) S.Labasse / labasse@toto (erreur)	Arrange, Act, Assert

TESTS

✓ Plan de test

→ Fonctionnels

Playwright

Synthèse

COMPARAISON

Unitaire

- Classe
- Test des appels de méthode
- Cas usuels, extrêmes, erreur
- Dépendances simulées
- Environnement neutre

Fonctionnel

- Programme
- Test des actions utilisateur
- Cas usuels, extrêmes, erreur
- Dépendances réelles
- Environnement similaire à la production

OUTILS

Web (OpenSource)

Selenium Web Driver via Http

Microsoft Playwright via WebSocket

TestCafe via proxy qui instrumente la page

Autres

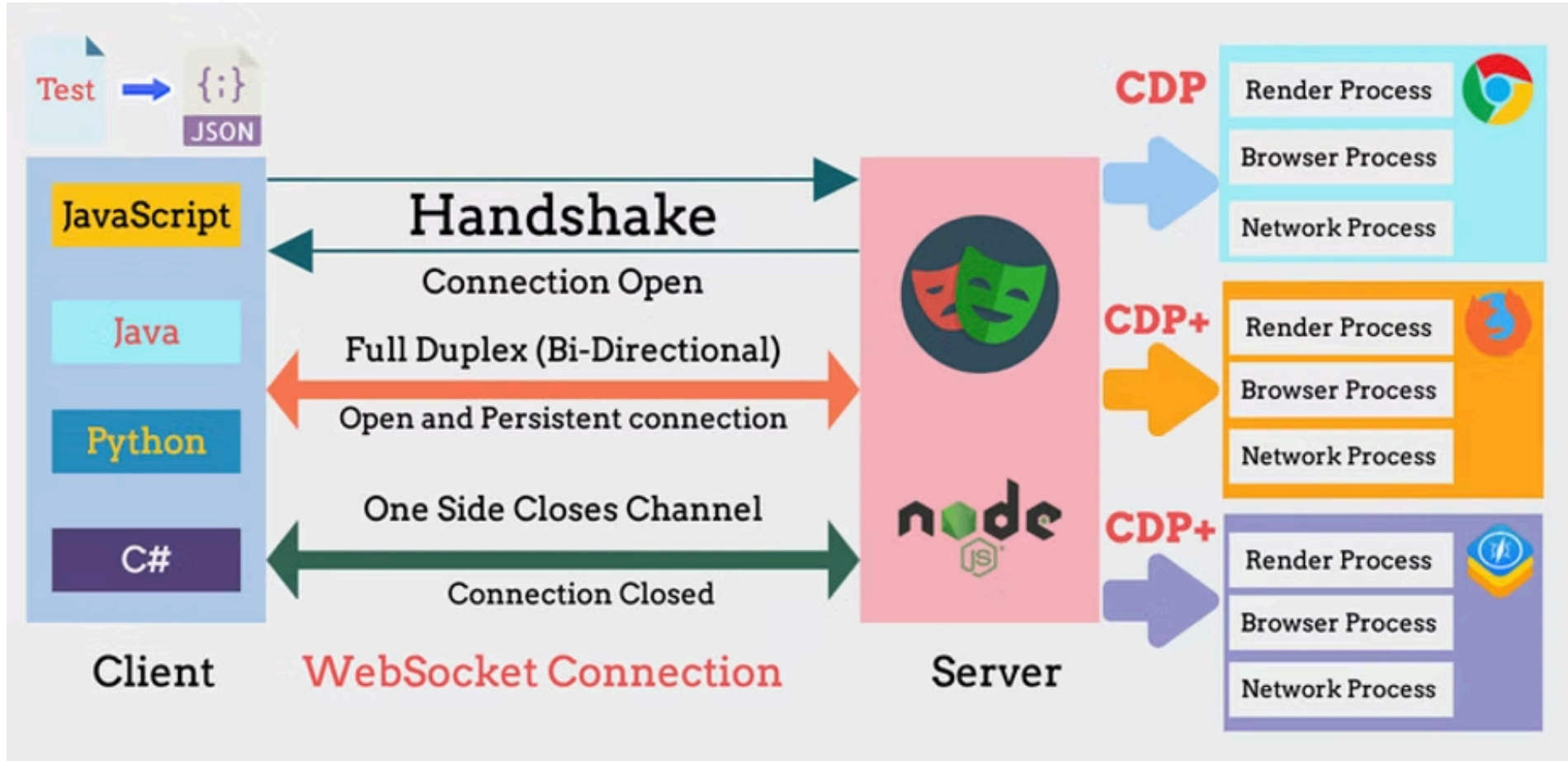
Mobile : Appium, Espresso , XCUITest , Detox (ReactNative)

Robotic Process App : RobotFramework (open source), UiPath

TESTS

- ✓ Plan de test
- ✓ Fonctionnels
- Playwright
- Synthèse

ARCHITECTURE PLAYWRIGHT



SYNTAXE PLAYWRIGHT

(voir exemple)

TESTS

- ✓ Plan de test
- ✓ Fonctionnels
- ✓ Playwright
- Synthèse

RÉSUMÉ

Utilité des tests pour la maintenabilité

Construction et contenu d'un plan de test

Outils de tests fonctionnels selon le type d'appli.

Prise en main de Playwright pour le web et les APIs

DEVSECOPS

OBJECTIFS

Ciblage de vecteurs d'attaque via STRIDE

Scan statique, dynamique et interactif

Seuil pour les vulnérabilités critiques

Cartographie d'application via un SBOM

Prévention de divulgation de secrets

DEVSECOPS

→ Modélisation des menaces

Analyse statique

Supply Chain

Synthèse

MENACES ?

Prévention vs Réaction

Failles potentielles dès la conception
Coûts de correction min. (loi de Boehm)

Zones de confiance

Modélisation de l'architecture

Tracé des *Trust boundaries*

Ex : Entre utilisateur, application, base de données



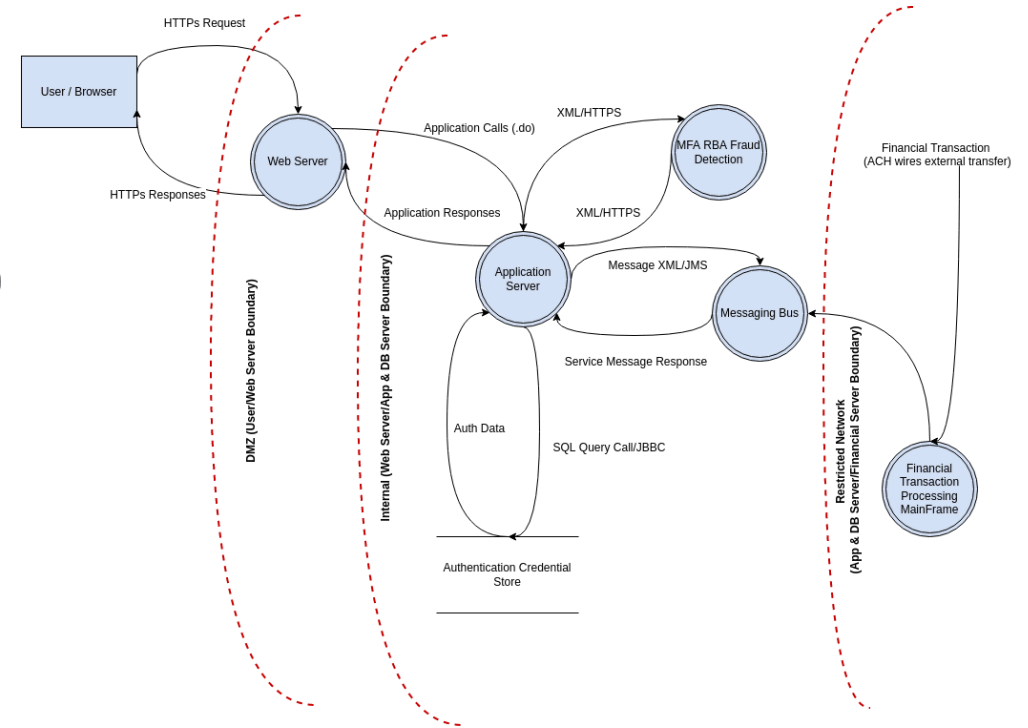
STRIDE

Menaces clés

S poofing : Usurpation
T ampering : Altération (données)
R épudiation
I nformations divulguées
D éni de service
E lévation de privilèges.

Exploitation du diagramme

1 menace = 1 ticket sécu. / "User Story" de sécurité (backlog)



DEVSECOPS

✓ Modélisation des menaces

→ Analyse statique

Supply Chain

Synthèse

LES DEUX PILIERS

Analyse statique (SAST)

Analyse sans exéc., détection précoce (Shift Left) failles logiques

⚠ Problèmes de configuration serveur non détectés

🔧 [SonarQube](#) (simple), [Semgrep](#) (SAST, DAST, SCA)

Analyse dynamique (DAST)

Test en exécution (boîte noire), failles session, chiffrement, config. runtime, ...

⚠ Tard dans le cycle, nécessite un environnement stable.

🔧 [Zed Attack Proxy ZAP](#) (Web), [StackHawk](#) (CI/CD), [Nikto](#) (conf)

«TROUS DANS LA RAQUETTE»

Interactive Analysis (IAST)

Agents installés dans l'application => spécifique au langage

Corrélation du code (SAST) et du trafic réseau (DAST)

Précision accrue + quasi-absence de faux positifs

 Contrast Security Community Edition

Fuzzing (Test de robustesse)

Injection de données malformées/aléatoires => plantage appli.

Failles d'overflow ou de déni de service

 **AFL++** (American Fuzzy Lop)

DEVSECOPS

- ✓ Modélisation des menaces
- ✓ Analyse statique
- Supply Chain
- Synthèse

SBOM

Software Bill Of Material

Inventaire des dépendances

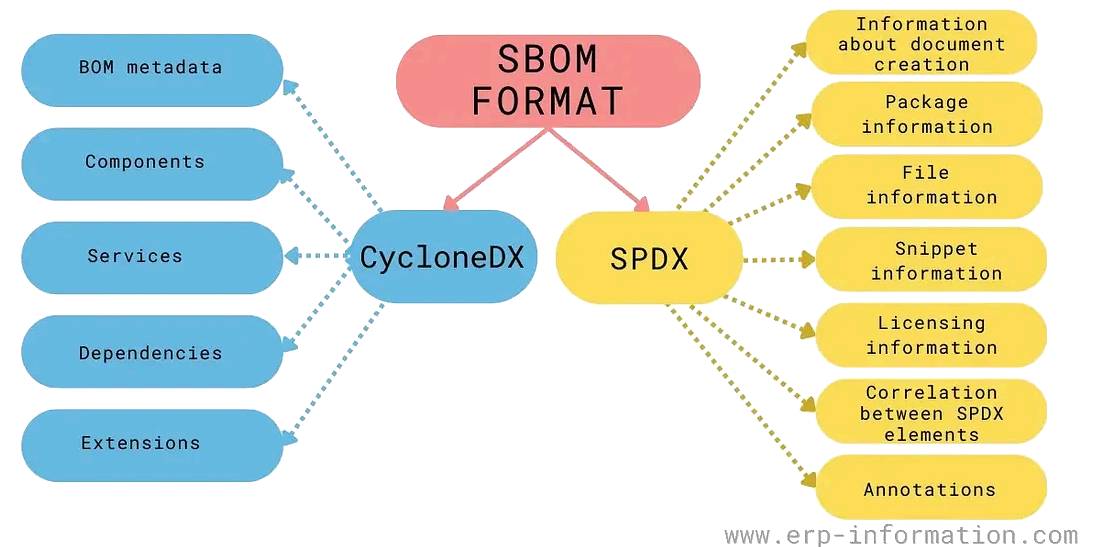
Composants et modules

Transparence « ingrédients »

Formats standards

CycloneDX : Léger, conçu pour l'automatisation et la sécurité

SPDX : Standard ISO, très utilisé pour la conformité des licences



DÉPENDANCES

Directes et transitives

1 librairie importée en cache 50

Risque souvent caché dans l'arborescence des dépendances

Attaques courantes

Typosquatting : nom proche

Ex : `pip install request` / `pip install requests`

Dependency Confusion : Interne `epsi.xxx` / Public infecté

SECRETS

Divulgations

Code : « en clair », `.env` non ignoré, ou ignoré tard (historique)

Logs : Historique du CI/CD

Bdd : Clair ou hash fragile

Git n'oublie rien

Suppression inutile si historisé dans `.git`

Secret Scanning possible avec *Gitleaks* ou *TruffleHog*

DEVSECOPS

- ✓ Modélisation des menaces
- ✓ Analyse statique
- ✓ Supply Chain
- Synthèse

RÉSUMÉ

Ciblage de vecteurs d'attaque via STRIDE

Scan statique, dynamique et interactif

Seuil pour les vulnérabilités critiques

Cartographie d'application via un SBOM

Prévention de divulgation de secrets

BILAN

VOUS SAVEZ MAINTENANT...

Organiser une équipe de dév. pour la maintenance

Utiliser des outils adaptés pour assurer la qualité

Prévenir les régressions via les tests fonctionnels

Sécuriser la supply chain par une approche DevSecOps

Consolider progressivement l'application